

Master in Management & Analytics



75/100

Reinforcement Learning

Year 2025

Students

Juan Ignacio Galván

Marcelo Vera Caballero

Propuesta del Problema a Resolver

El problema que proponemos resolver es la gestión óptima de reservas de un restaurante que posee dos tipos de menú fijo de tres pasos pero de diferentes precios (Premium y Estándar), y enfrenta la decisión de aceptar o rechazar reservas provenientes de clientes que, al efectuar su reserva, deben indicar tanto el tipo de menú como la cantidad de personas (hasta un máximo de 4).

Para simplificar la modelización y mantener la complejidad en un nivel manejable, agrupamos todas las posibles combinaciones de menú y tamaño de grupo en 4 segmentos (alto premium, bajo premium, alto estándar, bajo estándar), cada uno con un ingreso promedio predefinido. Esta categorización refleja el impacto conjunto del menú y del tamaño de grupo, sin modelar cada combinación de manera individual.

El objetivo principal es maximizar la rentabilidad total obtenida durante un período específico (una noche), considerando que existe un número limitado de mesas disponibles y una cantidad determinada de oportunidades de reservas. Adicionalmente, se incorporan fenómenos realistas tales como la posibilidad de cancelaciones y la probabilidad de no-show (reservas confirmadas cuyos clientes finalmente no se presentan), en cuyo caso la mesa ya no se recupera. También se contempla una penalización asociada al rechazo de reservas.

Se adoptaron estas extensiones con el fin de enriquecer el modelo y acercarlo más a la operativa real de un restaurante, manteniendo la complejidad en un nivel manejable.

1. Modelando el problema

Modelo inicial de MDP

Variables Estado:

- m : número de mesas disponibles para reservar ($m \in \{0, 1, \dots, M\}$)
- t : oportunidades de reservas restantes ($t \in \{0, 1, \dots, T\}$, donde T es el número máximo de reservas que puede haber en la cola)
- r_p, r_e : reservas aceptadas de menú premium y estándar, respectivamente.

En cada oportunidad de reserva se observa el segmento (combinación de menú y tamaño de grupo) de la reserva entrante, pero no se modela el tamaño de grupo ni el tipo de menú

como variables de estado individuales. El estado evoluciona según la decisión tomada y el evento que ocurra (asistencia, cancelación, no-show).

Acciones:

- Aceptar la reserva ($a = 1$), si $m > 0$
- Rechazar la reserva ($a = 0$)

Dinámica del sistema

En cada paso de la noche, el sistema recibe el pedido de una nueva reserva (categorizada en uno de los 4 segmentos de reservas en base al producto de cuántas personas y qué menú eligen, tipo de segmento aleatorio según probabilidades conocidas) Al tomar una acción:

- Si se acepta la reserva:
 - Se reduce en 1 el número de mesas disponibles ($m' = m - 1$).
 - Se incrementa rp o re en 1, según el menú elegido ($rp' = rp + 1$ si es premium, $re' = re + 1$ si es estándar).
 - Se reduce el contador de reservas pendientes ($t' = t - 1$).
 - Con probabilidad, pueden ocurrir cancelaciones o no-shows, que no liberan nuevamente las mesas y generan penalizaciones.
- Si se rechaza la reserva:
 - rp y re permanecen iguales.
 - Se reduce el contador de reservas pendientes ($t' = t - 1$).
 - Penalización: $r = -P_{\text{rechazo}}$ (por ejemplo, pérdida de reputación o potenciales ingresos futuros)
- El proceso termina cuando $t = 0$ o $m = 0$.

Una vez aceptada una reserva, pueden contabilizar la recompensa inmediata basada en el tipo de cliente y probabilidad de que no cancele. Esa información no se vuelve a usar, entonces no es necesario mantener por separado el número de mesas de cada segmento en el estado, solo el número restante de mesas libres.

Recompensa

Acá hay 4 segmentos per el estado distingue solo entre reserva premium y standard?

- Si la reserva es aceptada y el cliente asiste:
 - Si es alto premium: $r =$ ingreso fijo que representa el valor del segmento(INGRESO_ALTO_PREMIUM).

- Si es bajo premium: $r =$ ingreso fijo que representa el valor del segmento.(INGRESO_BAJO_PREMIUM)
- Si es alto estándar: $r =$ ingreso fijo que representa el valor del segmento.(INGRESO_ALTO_ESTANDAR)
- Si es bajo estándar: $r =$ ingreso fijo que representa el valor del segmento.(INGRESO_BAJO_ESTANDAR)
- Si hay cancelación previa (antes del servicio):
 - Se puede adicionar una penalidad: $r = -P_{cancel}$
- Si hay no-show (el cliente no se presenta la noche del servicio):
 - Se aplica una penalidad específica: $r = -P_{no-show}$

Condición de suficiencia

El proceso termina cuando $t = 0$, es decir, cuando no quedan más oportunidades de reservas para procesar durante la noche. Aunque puede haber mesas libres (por cancelaciones o no-shows), una vez agotadas las oportunidades de reserva no se reciben nuevos pedidos.

Objetivo

Maximizar la rentabilidad esperada del restaurante en una noche, considerando los ingresos diferenciales por tipo de menú, las penalizaciones asociadas a cancelaciones, no-shows y rechazo de reservas.

Justificación del modelo

Nuestro modelo utiliza un MDP donde el estado incluye la cantidad de mesas libres, las reservas restantes y las reservas aceptadas premium y estándar. En cada paso, se observa la llegada de una nueva reserva (siempre perteneciente a un segmento, el tipo de menú y el segmento se conoce), y se debe decidir si aceptarla o rechazarla. El modelo representa la incertidumbre a través de la asignación probabilística de segmento (alto premium, bajo premium, alto estándar o bajo estándar), y contempla fenómenos estocásticos realistas como cancelaciones y no-shows sobre reservas aceptadas. La dinámica temporal está dada por la evolución secuencial de las reservas a lo largo de la noche, el cambio de estado según las decisiones tomadas y los eventos aleatorios que pueden generar pérdidas tras una cancelación o no-show. El proceso termina cuando se agotan las oportunidades de

recibir reservas. El modelo busca maximizar la rentabilidad esperada de la noche, considerando los ingresos diferenciados por menú y penalizaciones por rechazo, cancelaciones y no-shows. Elegimos el modelo para reflejar las decisiones realmente tomadas en un restaurante, manteniendo la estructura lo suficientemente simple para aplicar algoritmos exactos y de refuerzo. La principal simplificación es que todas las reservas se categorizan dentro de 4 segmentos y ocupan una sola mesa, lo que facilita la representación y solución, y elimina la complejidad que supondría manejar grupos de diferente tamaño.

2. Implementación numérica en Python

```
def f(x, a, V, tabla):
    sub = tabla[(tabla['Estado'] == x) & (tabla['accion'] == a)]
    valor_esperado = 0.0
    for _, row in sub.iterrows():
        p = row['probabilidad']
        r = row['recompensa']
        y = row['prox_estado']
        Vy = V.get(y, 0.0)
        valor_esperado += p * (r + alpha * Vy)
    return valor_esperado
```

```
def a_estrella(x, V, tabla):
    valores = [f(x, a, V, tabla) for a in acciones]
    return np.argmax(valores), max(valores)
```

```
def g(x, a, tabla):
    sub = tabla[(tabla['Estado'] == x) & (tabla['accion'] == a)]
    probabilidades = sub['probabilidad'].to_numpy()
    if probabilidades.sum() == 0:
        return 0, x # Estado terminal
    elegido = np.random.choice(sub.index, p=probabilidades/probabilidades.sum())
    row = sub.loc[elegido]
    return row['recompensa'], row['prox_estado']
```

Siendo en nuestro código la variable **tabla** una composición de todos los estados y acciones posibles. Adjuntamos link de nuestro [Google Colab](#).

3. Algoritmos de solución

La aplicación de los tres algoritmos de solución se pueden ver en nuestro [Google Colab](#).

En nuestra implementación, iteración de política fue el algoritmo exacto más rápido en converger; la iteración de valor fue correcta pero un poco más lenta debido al criterio de convergencia punto a punto. Q-learning resultó considerablemente más lento y sus resultados fueron más variables entre corridas, necesitando miles de noches simuladas para aproximarse al rendimiento y política óptima hallada por los métodos exactos. ✓

Al comparar los resultados, vimos que tanto iteración de valor como iteración de política obtienen la política verdadera óptima (idéntico valor esperado y decisiones óptimas), mientras que Q-learning se acerca a esa política sólo tras suficiente exploración y aprendizaje, y su desempeño depende del balance adecuado entre exploración (ϵ) y explotación, así como de la tasa de aprendizaje (α). ✓

En cuanto a la tasa de aprendizaje de nuestro algoritmo Q-learning, lo que hicimos fue ir variando el mismo. Comenzamos colocando una tasa fija de 1 pero viendo los ejemplos en clase, notamos que en algunos casos (como el ejemplo de la clase 4) se utilizaba un learning rate más adaptativo y dependiente del episodio en el que se estaba. Esto quiere decir que a medida que iban pasando los episodios, la tasa de aprendizaje era cada vez menor, buscando que sea más preciso en las iteraciones finales.

Por otro lado, el epsilon en nuestro modelo es importante ya que:

- En términos de exploración permite no quedarse atrapado en una política subóptima inicial. ✓
- En términos de explotación permite usar la mejor información aprendida hasta ese momento para maximizar la recompensa. ✓

Cada vez que aparece `if np.random.rand() < EPSILON`, el agente decide entre explorar o explotar. Al principio del entrenamiento conviene que `EPSILON` sea alto: más exploración. Conforme aprende, se puede disminuir `EPSILON` para explotar más lo aprendido.

A nuestro criterio, los resultados obtenidos en lo que corresponde a la política de value iteration o policy iteration no son los mejores porque opera siempre de la misma manera y

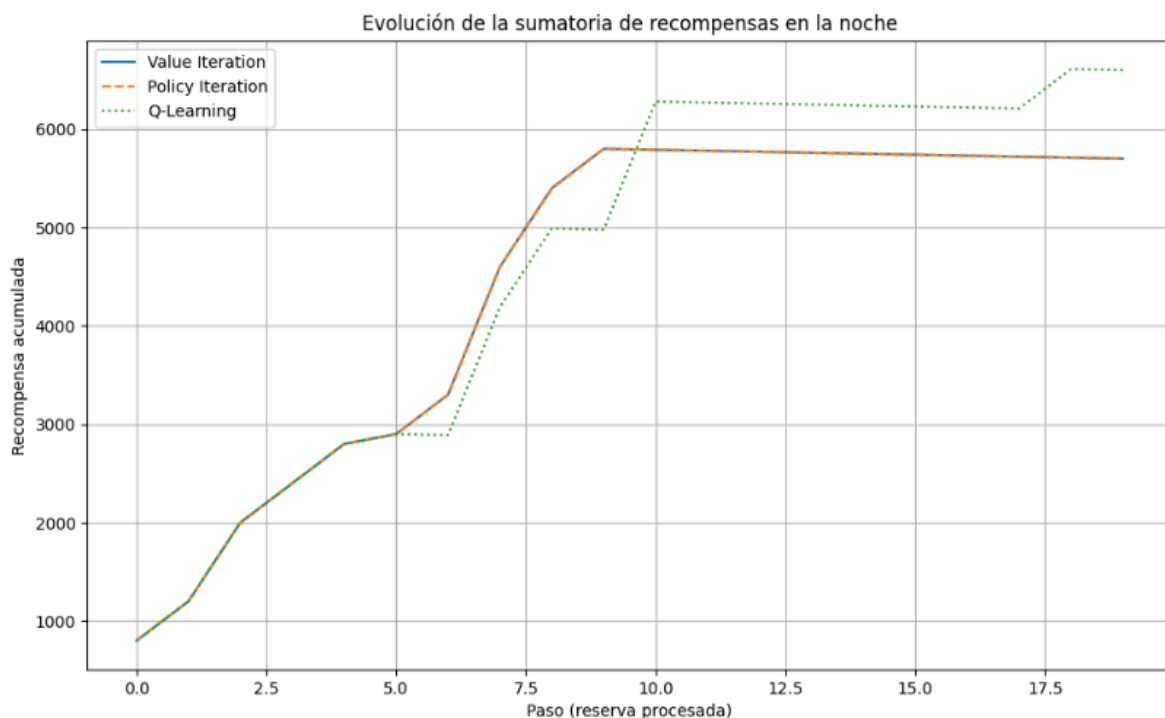
nunca pudimos revertir la situación en la que el algoritmo acepte a los primeros 10 clientes.

Modificamos parámetros como las recompensas, las penalizaciones por no-show o cancelaciones, las penalidades por cancelar y aún así siempre el modelo para value y policy iteration arribó a las mismas conclusiones.

Abajo relatan que el algoritmo acepta primero reservas premium, entonces en ese primer momento no está rechazando reservas estandar?

4. Análisis de resultados

Análisis de políticas óptimas obtenidas



Las políticas óptimas obtenidas con iteración de valor y de política son prácticamente iguales y muy consistentes: ambas buscan maximizar la rentabilidad dando prioridad a las reservas premium, que aportan más ingreso, y asignan mesas a los clientes estándar cuando ya no hay demandas más valiosas. Cuando se llenan todas las mesas, la política automáticamente comienza a rechazar las reservas, lo cual es lógico porque aceptar ya no sería posible y rechazar solo implica la penalización de rechazo, que es menor que cualquier posible ingreso.

Q-learning, por su parte, logra imitar esta política después de mucho entrenamiento, aunque puede mostrar algunas variaciones en trayectorias individuales debido a su componente

aleatorio y a la exploración que hace durante el aprendizaje. Aun así, cuando Q-learning está bien configurado y con suficientes noches de simulación, su política converge a un comportamiento estratégico muy similar: prioriza premium, acepta estándar cuando corresponde, y rechaza cuando ya no hay más lugar.

En todos los casos, las decisiones que toma el sistema reflejan que el modelo está bien ajustado a los objetivos del restaurante: aprovechar al máximo la capacidad disponible y minimizar las pérdidas asociadas a rechazos, no-shows y cancelaciones.

Impacto del modelo y complejidad computacional

La complejidad computacional del modelo depende directamente de las decisiones de modelado, en especial la cantidad de variables de estado y la granularidad con que se representa la dinámica del sistema. En nuestro caso, al incluir distinciones entre menú premium A y B/estándar A y B (con diferentes precios), la contabilidad de reservas previas, y la posibilidad de cancelaciones y no-shows, el espacio de estados crece rápidamente. Esto implica que los algoritmos exactos como iteración de valor o de política requieren almacenar y actualizar grandes tablas de valores, lo que aumenta notablemente tanto la memoria necesaria como el tiempo de cómputo a medida que crece el número de mesas o reservas.

Por eso, aunque el modelado detallado permite políticas óptimas más realistas, también puede volver la resolución inabordable en problemas de mayor tamaño. Métodos aproximados como Q-learning alivian este problema ya que no exploran cada estado de manera exhaustiva, pero requieren muchos episodios de simulación y su rendimiento puede ser más variable. En síntesis, cuanto más realista y detallado es el modelo, más importante es prestar atención a la gestión de los recursos computacionales y considerar la escalabilidad de los algoritmos elegidos.

Necesitamos aproximaciones de la función valor o Q para aliviar la maldición de la dimensionalidad.

Direcciones futuras del modelo

Gestión de sobreventa (overbooking)

Una extensión relevante del modelo sería permitir overbooking (aceptar más reservas que mesas disponibles) para compensar cancelaciones y no-shows. El desafío principal radica en balancear ingresos adicionales con el riesgo de tener más clientes presentes que mesas, lo que podría generar altos costos por compensaciones o pérdida de reputación. Para moldearlo, se deberían agregar penalizaciones si el número de clientes asistentes supera la capacidad disponible. Esta extensión haría el modelo más realista en escenarios de alta demanda, pero aumentaría la complejidad tanto en la formulación como en la solución óptima. ✓

Penalización dependiente del historial y la fidelización con el cliente

Una posible extensión interesante del modelo es que la penalización por rechazar reservas dependa del historial y nivel de fidelización de cada cliente. Así, rechazar a un cliente frecuente implicaría una penalización mayor que hacerlo con un cliente nuevo, reflejando la importancia de la relación de largo plazo y el valor de vida del cliente para el restaurante. Esta variación incentiva políticas que prioricen la aceptación de reservas de clientes valiosos o habituales. Implementar esta idea requeriría conocer o estimar el historial de cada cliente y actualizarlo en el estado del sistema, lo que aumentaría la complejidad del modelo pero lo haría más realista y estratégico. ✓

Mayores simulaciones de demanda

Una posible extensión también podría haber sido simular más veces la demanda y ver si en un promedio de 10 días hacia atrás, con distintas probabilidades para distintas demandas, los algoritmos de value iteration o policy iteration podrían tener algún rechazo en los pedidos que llegaban. En este sentido, podría asemejarse más a una situación real y no ser algo de una sola noche. Requeriría un poder de cómputo mucho mayor dado que incrementa 10 veces las distintas variables estado y sus acciones. ✓